WPT Day 11

Sat 31 Oct 26 · React with Vite (project setup and boilerplate cleanup), class vs function components, JSX (JavaScript + XML), props, state (`this.setState` / `useState`), event handling, the Virtual DOM, hooks, the class component lifecycle (`constructor` → `render` → `componentDidMount`), fetching data in `componentDidMount` , list rendering with `.map()` , conditional rendering, and a short aside on MCP

Table of Contents

1. React App Setup with Vite and Cleaning the Boilerplate
2. Components: Class vs Function
3. JSX: JavaScript + XML
4. Props: Passing Data into a Component
5. State: Data a Component Owns
6. Event Handling in React
7. Virtual DOM
8. Hooks
9. Component Lifecycle
10. Fetching Data in componentDidMount
11. List Rendering with map()
12. Conditional Rendering and the Dry Run
13. MCP (Model Context Protocol): A Brief Aside
14. Combined Worked Example
15. Quick Revision Checklist
16. Practice Exercises
17. Practice Answers

React App Setup with Vite and Cleaning the Boilerplate

Why this concept exists

Yesterday we saw that a React app is a single-page application (SPA) that needs a build tool to turn `.jsx` files into something a browser can run. Writing that tooling by hand is painful, so a **scaffolding tool** (Vite) generates a ready-made project. Today starts by creating a fresh app and then removing everything we do not need, so that we begin with a **clean, plain React app with no predefined styles**.

Mechanism: creating the project

```
npm create vite@latest
```

Vite then prompts for a few answers:

Prompt	Lecture's answer
Project name	`React Demo`
Framework	`React`
Variant	`JavaScript`
ESLint	selected

```
# Install dependencies after creation (inside the project folder)
npm i

# Start the development server
npm run dev
```

- `npm i` generates the **`node_modules`** folder and **`package-lock.json`**.
- `npm run dev` starts the dev server; the app launches on **`localhost:5173`**.

Background: `package.json` lists what the project needs; `npm i` reads it, downloads every package into `node_modules`, and records the exact versions in `package-lock.json`. A project is shipped without `node_modules` because anyone can regenerate it with `npm i`.

Cleaning the boilerplate

- Vite generates default CSS (`App.css`, `index.css`) — these **belong to Vite, not React itself**.
- Remove all default content from `App.jsx` and replace it with a minimal structure:

```
function App() {
  return (
    <div>
      <h1>Welcome to our app</h1>
    </div>
  );
}

export default App;
```

- Also remove the CSS import from `main.jsx`:

```
// Remove this line:
import './index.css'
```

> **Exam:** The default CSS files are generated by **Vite**, not by React. Cleaning the boilerplate means: empty `App.jsx` down to a minimal component, and delete the `index.css` import in `main.jsx`.

Components: Class vs Function

Problem

Everything visible in a React app is a **component**. Yesterday we met the idea; today we write them. React offers **two ways** to create a component: a **class component** and a **function component**.

Class components

A plain JavaScript class is **not** a React component by itself. It must **inherit from the `Component` class provided by React**, which is where React-specific abilities (`render`, `this.props`, `this.state`, lifecycle) come from.

```
import { Component } from 'react';
```

Three mandatory steps:

1. Define a class and `extend Component`.
2. Define a `render()` function that returns the UI.
3. Export the class.

Navbar example (lecture):

```jsx
// navbar.jsx
import { Component } from 'react';

export class Navbar extends Component {
  render() {
    return (
      <div>
        <a href="#">Home</a>
        <a href="#">About Us</a>
        <a href="#">Contact Us</a>
      </div>
    );
  }
}
```

Footer example (lecture):

```jsx
// footer.jsx
import { Component } from 'react';

export class Footer extends Component {
  render() {
    return (
      <div>
        <h4>This is the footer</h4>
      </div>
    );
  }
}
```

Using components inside App

- Import **without** the `.jsx` extension (React/Vite is already configured to resolve `.js` and `.jsx`):

```jsx
import { Navbar } from './navbar';
import { Footer } from './footer';
```

- Every component gets a **corresponding tag matching its class name**:

```jsx
function App() {
  return (
    <div>
      <Navbar />
      <h1>Welcome to our app</h1>
      <Footer />
    </div>
  );
}

export default App;
```

- React calls `render()` behind the scenes whenever the component is displayed — you never call it yourself.

> **Exam:** Key rule: to use any component (class or function), use its **corresponding tag**. The tag name matches the class/function name **exactly** (`Navbar` → `<Navbar />`).

Function components

The simpler alternative: **just a function that directly returns UI**. No class, no inheritance, no separate `render()` — the function itself *is* the render.

```jsx
// navigationbar.jsx
export function NavigationBar() {
  return (
    <div>
      <a href="#">Home</a>
      <a href="#">About Us</a>
      <a href="#">Contact Us</a>
    </div>
  );
}
```

It is used **exactly the same way** as a class component:

```jsx
<NavigationBar />
```

- **Convention:** keep the **file name, function name and component name the same** so you do not track two different names. This is not a hard requirement — only readability.
- Function components are lightweight: no class overhead, no inheritance, no `render()`.
- **Preferred today** for better performance; use class components only if the team strictly enforces OOP principles.
- Function parameters in components are called **props**, not "parameters" — covered in the Props section.

Class vs function at a glance

Aspect	Class component	Function component
Definition	`class X extends Component`	`function X() { ... }`
UI provided by	`render()` method	The function body's `return`
Inheritance	Yes — from `Component`	None
Receives props via	`this.props`	Function argument
State	Native (`this.state` , `this.setState`)	Via `useState` hook
Lifecycle	Native (`componentDidMount` , ...)	Via `useEffect` hook
Weight / speed	Heavier	Lightweight, faster

Aspect	Class component	Function component
Use today	Only if team enforces OOP	Preferred

JSX: JavaScript + XML

Problem

In a normal `.js` file, you cannot write `<div> ... </div>` directly — it is a syntax error. Yet React components need to describe UI in an HTML-like form. We need a way to **mix HTML-looking code into JavaScript**.

What is JSX?

- **JSX = JavaScript + XML.**
- It is a file extension (`.jsx`) that lets a developer write HTML code directly inside JavaScript.
- A regular `.js` file cannot do that; JSX makes it possible by combining the two.

HTML vs XML

	HTML	XML
Tags	**Predefined** (`<div>` , `<p>` , `<img>`)	**User-defined only** — no predefined tags
Purpose	Display data on a web page	**Hold and store data** in tag format (used before JSON became standard)

XML example (lecture):

```
<student>
  <id>101</id>
  <name>Shivam</name>
</student>
```

- React uses XML behind the scenes to generate **custom / user-defined tags** for your components.
- `<Navbar />` , `<Footer />` , `<Dashboard />` are **not HTML tags** — they are XML-style user-defined tags.
- That is why the extension is `.jsx` (JavaScript + XML).

Rule 1: Every tag must be closed

XML rule: every opened tag must be closed — React **inherits** this rule. It applies to predefined HTML tags and custom component tags alike.

```
// Wrong in React:
<br>
<input>

// Correct in React:
<br />
<input />
// or
<br></br>
```

The self-closing shorthand (slash at the end of the opening tag) is valid and is commonly used for tags with no inner content.

Rule 2: Single parent

JSX expressions must have **one parent element**. Returning multiple siblings at the same level causes an error:

```
// Wrong — two siblings at the same level:
return (
  <div><a href="#">Home</a></div>
  <img src="logo.png" />
);
// Error: JSX expressions must have one parent element
```

Background: a function can return only one value, and each JSX tag turns into one JavaScript expression/object. Two sibling tags would mean returning two values, which is impossible — hence one wrapper.

Fix 1 — wrap in a single parent `<div>`:

```
return (
  <div>
    <div><a href="#">Home</a></div>
    <img src="logo.png" />
  </div>
);
```

Fix 2 — an empty fragment `<> … </>` (adds no extra DOM node):

```
return (
  <>
    <div><a href="#">Home</a></div>
    <img src="logo.png" />
  </>
);
```

Wrapper	Extra element in DOM?
`<div> … </div>`	Yes — an extra `div`
`<> … </>` (fragment)	No

> **Pitfall:** `<br>` and `<input>` without a closing slash work in plain HTML but are **errors in JSX**. Always write `<br />` and `<input />`.

Props: Passing Data into a Component

What are props?

- **Props = attributes of a component** — React's term for what HTML calls attributes.
- In HTML: `<img src=" … " width=" … " />` — `src` and `width` are attributes.
- In React the same things are called **props**.
- For predefined tags, the props are already defined (`src` , `href` , `align` , ...). For **custom components, the developer defines their own props**.

Problem: the reusable Intro component

Scenario: the Home, About and Contact pages all share the same layout section — a heading, a paragraph and a button — but with different content. Repeating 20–25 lines of HTML in every component is wasteful.

The fix: create **one reusable `Intro` component**. It knows its **structure** (`h1` + `p` + `button`) but **cannot decide the content** — that varies per page. This is exactly when props are used: to pass dynamic data into a component from outside.

```
// In App (or any parent component):
<Intro
  headingText="Welcome to our app"
  description="This is our first React app"
```

```
  buttonText="Click Me"
/>

// Rendered again from a different context:
<Intro
  headingText="Know About Us"
  description="We have 20 years of experience in the industry"
  buttonText="Know More About Us"
/>
```

Two key rules about props

1. A component **cannot decide the value of its own props** — the outside world (the parent component) decides.
2. A component **should not change** the value of its props — props are **read-only**.

If a component could decide or change its own props, there would be no need for props at all.

Props in a function component

React bundles all passed props into **a single object** and passes it as the argument of the function:

```
Behind the scenes React creates:
{ headingText: "Welcome to our app",
  description:  "This is our first React app",
  buttonText:   "Click Me" }
```

Receive and use them:

```
export function Intro(props) {
  return (
    <div>
      <h1>{props.headingText}</h1>
      <p>{props.description}</p>
      <button>{props.buttonText}</button>
    </div>
  );
}
```

Destructuring for cleaner syntax (recommended):

```
export function Intro({ headingText, description, buttonText }) {
  return (
    <div>
      <h1>{headingText}</h1>
      <p>{description}</p>
      <button>{buttonText}</button>
```

```
    </div>
  );
}
```

Curly braces `{}` in JSX embed any JavaScript variable or property value into the HTML.

- Without curly braces, the variable name prints as **literal text** on screen (`<h1>headingText</h1>` shows the word "headingText").
- React uses `react-dom` internally to perform the DOM manipulation and display the value.

Props in a class component

A class has no function arguments, so props become a **class-level member property**. React creates the same props object and assigns it to `this.props` (inherited from `Component`).

```
// introduction.jsx
import { Component } from 'react';

export class Introduction extends Component {
  render() {
    const { headingText, description, buttonText } = this.props;
    return (
      <div>
        <h1>{headingText}</h1>
        <p>{description}</p>
        <button>{buttonText}</button>
      </div>
    );
  }
}
```

Passing props to a class component is identical to a function component — only **receiving** differs:

Component type	Receives props via
Function component	The function **argument**
Class component	`this.props` (inherited class-level property)

Passing data between components

Props are also the mechanism to pass data from one component to another. Example: `App` passes data **down** to `Intro` via props; `Intro` displays it.

> **Exam:** Props = outside world decides, component cannot change them (read-only). They flow from parent → child.

State: Data a Component Owns

What is state?

- **State = a property that a component owns, initializes, and can change itself.**
- It is the **opposite of props**:

	Props	State
Who decides the value?	The **outside world** (parent)	The **component itself**
Can the component change it?	**No** (read-only)	**Yes**, at any time
Use when	Data is supplied from outside	Data **changes over time** inside the component

Problem: the scoreboard

A scoreboard starts at **0**, **increases by 3** on a correct answer and **decreases by 1** on an incorrect one. The score changes over time and the component itself manages it — so the score is **state**.

State in a class component (native feature)

State is a **native feature of class components**, available from day one in React. Initialise it in the **constructor** using `this.state`:

```jsx
// scoreboard.jsx
import { Component } from 'react';

export class Scoreboard extends Component {
  constructor() {
    super();
    this.state = { score: 0 };
  }

  increaseScore = () => {
    this.setState({ score: this.state.score + 3 });
  }
```

```
  decreaseScore = () => {
    this.setState({ score: this.state.score - 1 });
  }

  render() {
    return (
      <div>
        <h1>{this.state.score}</h1>
        <button onClick={this.increaseScore}>+</button>
        <button onClick={this.decreaseScore}>-</button>
      </div>
    );
  }
}
```

- **Reading** state: `this.state.score`
- **Updating** state: always use `this.setState()` — **never mutate directly**:

```
// Wrong — do not do this:
this.state.score = this.state.score + 3;

// Correct:
this.setState({ score: this.state.score + 3 });
```

`setState` does two things:

1. Updates the state value in memory.
2. **Triggers a re-render** of the component so the UI reflects the latest data.

Without `setState`, memory may change but **the UI stays stale**.

Background: `super()` calls the parent (`Component`) constructor — in a subclass you must call it before using `this`. The lecture's later examples write `constructor(props) { super(props); ... }`, which additionally hands `props` up to `Component` so `this.props` is usable inside the constructor. Also, `increaseScore = () => { ... }` is an arrow-function class property; arrow functions keep `this` pointing at the component, so `this.setState` works when React calls the handler on a click.

State in a function component (via a hook)

State is **not native** to function components. It needs the **`useState` hook** — a regular JavaScript function provided by React.

```
import { useState } from 'react';
```

- `useState` takes the **initial value** as its argument and **returns an array of two elements**:
  1. The **current state value**
  2. A **function to update** that state
- Destructure the returned array with meaningful names:

```
const [score, setScore] = useState(0);
```

- **Naming convention:** if the state is called `score`, the setter is called `setScore`.

Scorecard function component (lecture):

```
// scorecard.jsx
import { useState } from 'react';

export function Scorecard() {
  const [score, setScore] = useState(0);

  const increaseScore = () => {
    setScore(score + 3);
  };

  const decreaseScore = () => {
    setScore(score - 1);
  };

  return (
    <div>
      <h1>{score}</h1>
      <button onClick={increaseScore}>+</button>
      <button onClick={decreaseScore}>-</button>
    </div>
  );
}
```

- **No `this` keyword** needed in function components — everything is directly accessible.
- **Each state variable gets its own `useState` call and its own setter:**

```
const [score, setScore] = useState(0);
const [player, setPlayer] = useState('');
const [users, setUsers] = useState([]);
```

- Contrast with class components, where **all state lives in one `this.state` object**:

```
this.state = { score: 0, player: '', users: [] };
```

- A state update still **triggers a re-render** in function components, exactly as in class components.

Class vs function: state side by side

	Class component	Function component
Declare	`this.state = { score: 0 }` in the constructor	`const [score, setScore] = useState(0)`
Read	`this.state.score`	`score`
Update	`this.setState({ score: ... })`	`setScore( ... )`
Shape	One object for all state	One `useState` per variable
Re-render on update	Yes	Yes

> **Pitfall:** Writing `this.state.score = this.state.score + 3` changes memory but React is never told, so **no re-render** happens and the UI stays stale. Always use `setState` / the setter from `useState`.

Event Handling in React

Mechanism

- React uses **camelCase** for event props (they are props, not HTML attributes):

HTML	React
`onclick`	`onClick`
`onkeyup`	`onKeyUp`

- **Pass a reference to the function, not a call:**

```
// Wrong – this CALLS the function immediately on render:
<button onClick={this.increaseScore()}>+</button>

// Correct – passes the reference; React calls it on click:
<button onClick={this.increaseScore}>+</button>
```

- In **class components**, functions are class-level members and must be referenced with `this`:

```
onClick={this.increaseScore}
```

- In **function components**, functions are directly accessible without `this`:

```
onClick={increaseScore}
```

> **Pitfall:** `onClick={increaseScore()}` — with the brackets — runs the function **while rendering**, not on click. In a state-changing handler that also triggers another render, producing a loop. Pass `increaseScore` without `()`.

Virtual DOM

Problem

In plain HTML + JavaScript with direct DOM manipulation:

1. Any change to the DOM causes the **entire DOM tree** to be recreated from scratch.
2. Even changing **one element** forces the browser to repaint the whole document.

That is slow for pages that update often (a live score, a feed).

What is the Virtual DOM?

- React maintains a **virtual copy of the actual browser DOM**.
- It is **not a separate technology** — simply a **lightweight copy of the real DOM tree**.

Mechanism

```
1. React keeps the PREVIOUS virtual DOM snapshot
2. State updates  ⟶  React creates a NEW virtual DOM snapshot
3. React DIFFS the two (compares them) to find exactly what changed
4. ONLY the changed elements are updated in the ACTUAL DOM
```

- Result: far fewer DOM operations, much faster UI updates.
- **Example:** if only the score `<h1>` changed, only that element re-renders — not the Navbar, not the buttons.

Behind the scenes

- The `react-dom` library handles translating the virtual DOM into the actual browser DOM.
- **Future optimisation:** the `useMemo` and `useCallback` hooks can prevent unnecessary re-renders of child components that have not changed (**memoization** — covered later).

	Plain JS DOM manipulation	React with Virtual DOM
On a change	Entire DOM tree recreated	Compare old vs new virtual snapshot
What is updated	Whole document repaint	Only the changed elements
Speed	Slower	Much faster

> **Exam:** Virtual DOM = lightweight copy of the real DOM. React diffs the previous and new snapshot and updates **only what changed** in the real DOM.

Hooks

Problem

Function components are lightweight and preferred, but **state and lifecycle are native only to class components**. We need a way to give function components those abilities.

What are hooks?

- Hooks are **regular JavaScript functions** — nothing more.
- They exist to **bring class-native features** (state, lifecycle, etc.) **into function components**.
- **Analogy:** a hook "hangs" a class behaviour onto a function component.
- **Naming convention:** every hook name must **start with** `use`.
  - This is a **recommendation from React, not a hard syntax rule**.
  - It signals to anyone reading the code that this function is a hook.

Hook	Purpose
`useState`	Manages state
`useEffect`	Handles lifecycle / side effects
`useMemo`	Memoizes computed values

Hook	Purpose
`useCallback`	Memoizes functions
`useNavigate`	Navigation (React Router)
`useParams`	URL parameters (React Router)

- **Custom hooks** can also be written; they must also follow the `use` prefix convention.
- Any function in a React codebase whose name starts with `use` is a hook.

> **Exam:** Hooks are ordinary JS functions named `use ...` that give function components state (`useState`) and lifecycle (`useEffect`).

Component Lifecycle

Problem

A component is not static: it appears, may update many times, and is eventually removed. Code often needs to run **at specific moments** — "set things up when it first appears", "clean up when it disappears". We need named hook-points for those moments.

What is a lifecycle?

- Every component has a lifespan: from when it **first appears on screen** to when it is **removed**.
- **Lifecycle = the set of operations that happen to a component from start to end, in a defined sequence.**
- React provides **predefined functions** that are called **automatically** at each stage.

Two core phases

Phase	Meaning
Mounting	The component is displayed on screen for the first time (attached to the DOM)
Unmounting	The component is removed from the screen (e.g. the user navigates to a different page)

Lifecycle is a **native feature of class components** — the functions are inherited from `Component`. They are **not directly available in function components**, which need the `useEffect` hook (covered in the next session).

Lifecycle functions in a class component

1. `constructor()`

- Called **once**, during mounting, **before anything is displayed**.
- Responsible for **initialisation**: setting up `this.state`, binding methods, etc.
- `render()` **waits** for the constructor to finish. If the constructor takes 5 seconds, the screen stays blank for 5 seconds.

2. `render()`

- Called the first time during mounting, **immediately after the constructor** finishes.
- Called **again every time state is updated** (a re-render).
- Provides the UI for the component.

3. `componentDidMount()`

- Called automatically, **just once**, **immediately after the first render** completes.
- **Not** called on later re-renders caused by state updates.
- Useful for: **fetching data from an API**, setting up subscriptions, initialising third-party libraries.

4. `componentWillUnmount()` — called once when the component is removed (unmounting).

Console-log proof (lecture)

```
import { Component } from 'react';

export class Scoreboard extends Component {
  constructor() {
    super();
    this.state = { score: 0 };
    console.log('1. Constructor called');
  }

  componentDidMount() {
    console.log('3. componentDidMount called – only after first render');
  }

  increaseScore = () => {
    this.setState({ score: this.state.score + 3 });
  }

  decreaseScore = () => {
    this.setState({ score: this.state.score - 1 });
  }

  render() {
    console.log('2. Render called');
```

```
    return (
      <div>
        <h1>{this.state.score}</h1>
        <button onClick={this.increaseScore}>+</button>
        <button onClick={this.decreaseScore}>-</button>
      </div>
    );
  }
}
```

```
Console output on first load:
  1. Constructor called
  2. Render called
  3. componentDidMount called - only after first render

Console output after clicking "+" (state update):
  2. Render called          ⟵ constructor and componentDidMount do NOT run again
```

Lifecycle execution order summary

Stage	Function called	How many times
Mounting	`constructor()`	Once
Mounting	`render()`	Once (first time)
Mounting	`componentDidMount()`	Once (after first render)
State update	`render()`	Every state update
Unmounting	`componentWillUnmount()`	Once (on removal)

Lifecycle in function components

- State is not native → use the `useState` hook.
- Lifecycle is not native → use the `useEffect` hook.
- `useEffect` replicates `componentDidMount` and the other lifecycle methods — **next session**.

Why logs appear twice in development

If you run the code above in the dev server you may see each log **twice**.

- In `main.jsx`, React wraps the app in `<React.StrictMode>`.
- StrictMode **intentionally double-invokes** lifecycle methods in development to help catch side effects.
- This is **development-only**; in production each method runs exactly once.

- **Fix to see single logs:** open `main.jsx` and remove or comment out `<React.StrictMode>`.

```jsx
// main.jsx (StrictMode removed so each log appears once)
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <App />
);
```

Exam: Guaranteed order: **constructor** → **render** → **componentDidMount**. After that only `render()` fires on each state update. `componentDidMount` is a **one-time** event.

Pitfall: Seeing every log twice does **not** mean your code runs twice in production — it is `React.StrictMode` double-invoking in development only.

Fetching Data in componentDidMount

Problem: where should an API call go?

Many components must load data **as soon as they appear** (a feed, a users table). Which lifecycle function should hold the `fetch`?

Wrong place 1: the constructor

- The constructor runs **before** `render()`.
- An API call there **blocks** `render()` — the user sees a **completely blank screen**; not even a loader can appear because `render()` has not run.
- APIs can take 10–20 seconds on slow networks — a blank screen that long is unacceptable UX.

```jsx
constructor(props) {
  super(props);
  // BAD: API call here blocks render entirely
  fetch('https://api.example.com/data');
  // render waits, user sees nothing
}
```

Wrong place 2: `render()`

- `render()` 's only job is to **describe what the UI should look like**.

- It should be a **pure function**: same state/props → same output.
- An API call there creates **side effects and infinite loops** (a fetch → `setState` → re-render → fetch again ...).
- `render()` can be called **many times**; you do not want a new API call on every re-render.

The right place: `componentDidMount`

- It fires **after the first render has already painted the screen**.
- So the user already sees something: navbar, footer, static content, a **loader**, or a **shimmer / skeleton** effect.
- **Only then** does the data fetch begin in the background.
- When data arrives, `setState()` triggers a **second render** with the real data.

```
1. Constructor        → state = { users: [] }
2. First Render       → shows static UI + skeleton/loader (no data yet)
3. componentDidMount  → API call starts
4. API responds       → setState({ users: data })
5. Second Render      → shows actual data in the table
```

Real-world examples (lecture)

App	What you see first	Then
Instagram	Feed skeleton appears instantly	Posts load in
YouTube	Video card skeletons	Thumbnails and titles load
Facebook	Timeline skeletons	Posts populate
WhatsApp Web	Chat list shimmer	Actual chats load from the server

None of these apps ask you to click a button to "load the feed" — **data fetching starts the moment the component mounts**.

Other events that can trigger data fetching

Trigger	Example
Button click	User clicks "Search" or "Load More"
User typing	Autocomplete suggestions fetched as the user types
Component mount	The `componentDidMount` case — data loads automatically on page open

Full implementation: `UsersList` component

```jsx
// UsersList.jsx
import { Component } from 'react';

export class UsersList extends Component {
  constructor(props) {
    super(props);
    // Initialize state with an empty users array.
    // We don't have data yet, so start with []
    this.state = { users: [] };
    console.log("Inside constructor of UsersList");
  }

  async componentDidMount() {
    console.log("Inside componentDidMount of UsersList");
    try {
      // Step 1: Send GET request to the API
      const response = await fetch('https://jsonplaceholder.typicode.com/users');

      // Step 2: Parse the JSON body from the response
      const usersData = await response.json();

      // Step 3: Log the fetched data (for verification)
      console.log(usersData);

      // Step 4: Store the data in state to trigger a re-render
      this.setState({ users: usersData });
    } catch (error) {
      console.log(error);
    }
  }

  render() {
    console.log("Inside render of UsersList");
    return (
      <div>
        <h4>List of Users</h4>
        <table border="1" cellPadding="10" cellSpacing="0">
          <thead>
            <tr>
              <th>ID</th>
              <th>Name</th>
              <th>Username</th>
              <th>Phone</th>
              <th>Email</th>
            </tr>
          </thead>
          <tbody>
            {
```

```
              this.state.users.length === 0
                ? <tr><td colSpan="5">Loading ... </td></tr>
                : this.state.users.map(user => (
                    <tr key={user.id}>
                      <td>{user.id}</td>
                      <td>{user.name}</td>
                      <td>{user.username}</td>
                      <td>{user.phone}</td>
                      <td>{user.email}</td>
                    </tr>
                  ))
            }
          </tbody>
        </table>
      </div>
    );
  }
}
```

Changes to `App.jsx` (other components commented out temporarily):

```
// Comment out all other components temporarily
// import Intro from './Intro';
// import Introduction from './Introduction';
// import Footer from './Footer';
import { UsersList } from './UsersList';

function App() {
  return (
    <div>
      <UsersList />
    </div>
  );
}

export default App;
```

Why a local `const` cannot be used in `render()`

- Inside `componentDidMount`, `usersData` is declared with `const`.
- `const` and `let` are **block-scoped**: they exist only inside the `{}` block they are declared in.
- `render()` is a **completely different method** with its own scope, so it **cannot access** variables declared inside `componentDidMount`.

```
async componentDidMount() {
  const usersData = await response.json();   // lives only inside this block
}
```

```
render() {
  // usersData is NOT accessible here → ReferenceError
  return <div>{usersData}</div>;
}
```

State is the bridge

- State is attached to the **component instance** (`this.state`), not to any single method.
- **Any method** in the class can read from or write to `this.state` .
- `componentDidMount` **writes** the fetched data via `this.setState()` ; `render()` **reads** `this.state.users` .
- When `setState()` is called, React **automatically triggers a re-render**.

```
componentDidMount → this.setState({ users: data })
        |
        v
React triggers re-render
        |
        v
render() → reads this.state.users → displays table rows
```

> **Exam:** State is the only reliable way to share data between lifecycle methods in a class component. **Fetch in `componentDidMount` , store in state, read in `render` .**

List Rendering with map()

Problem: the old way

In vanilla JS, rendering a list meant creating and attaching every element by hand:

```
users.forEach(user => {
  const tr = document.createElement('tr');
  const td = document.createElement('td');
  td.innerHTML = user.name;
  tr.appendChild(td);
  tbody.appendChild(tr);
});
```

This is verbose, error-prone and hard to maintain: manual element creation, property setting and DOM appending.

The React way

React does not need you to create or append elements. It only needs **an array of JSX elements**:

1. Read your data array.
2. Use `.map()` to transform each data item into a JSX element.
3. Return the resulting array of JSX elements.
4. React reads that array and **renders all the elements automatically**.

Example 1: a simple array of numbers

```
// State
this.state = { myList: [10, 20, 30, 40, 50] };

// In render()
render() {
  return (
    <div>
      {
        this.state.myList.map(value => (
          <p>{value}</p>
        ))
      }
    </div>
  );
}
```

What `.map()` produces (conceptually):

```
[ <p>10</p>, <p>20</p>, <p>30</p>, <p>40</p>, <p>50</p> ]
```

React reads this array and renders five `<p>` tags automatically.

Example 2: an array of objects

```
// State
this.state = {
  myList: [
    { id: 111, name: 'Arun' },
    { id: 112, name: 'Rahul' },
    { id: 113, name: 'Suraj' }
  ]
};

// In render()
render() {
```

```
  return (
    <div>
      {
        this.state.myList.map(value => (
          <div key={value.id}>
            <h4>{value.id}</h4>
            <p>{value.name}</p>
          </div>
        ))
      }
    </div>
  );
}
```

What `.map()` produces (conceptually):

```
[ <div><h4>111</h4><p>Arun</p></div>,
  <div><h4>112</h4><p>Rahul</p></div>,
  <div><h4>113</h4><p>Suraj</p></div> ]
```

- For each object one `<div>` containing an `<h4>` (ID) and `<p>` (name) is created.
- React renders all three divs with **no manual DOM work**.
- No `for` loop — `.map()` handles iteration **and** array creation in one step.

Background: the `key={value.id}` prop (also `key={user.id}` in the users table) gives each list item a stable unique identity so React's virtual-DOM diff can tell which item changed, moved or was removed. Use a unique ID from the data. The lecture's first example omits it for brevity — React then shows a console warning.

	`forEach` + DOM calls	`.map()` in JSX
You create elements	Manually with `createElement`	Return JSX
You attach to the DOM	`appendChild`	React does it
Output	Nothing returned	**Array of JSX elements**

> **Exam:** React just needs an array of JSX elements. Use `.map()` to turn a data array into a JSX array — no `createElement`, no `appendChild`, no `innerHTML`.

Conditional Rendering and the Dry Run

Step-by-step dry run of `UsersList`

Step 1 — constructor runs:

- `this.state = { users: [] }` — the array is empty.
- No API call yet.

Step 2 — first `render()` runs:

- `this.state.users` is `[]`.
- `.map()` on an empty array produces nothing → `tbody` is completely empty.
- The user sees the table header but no rows.

Step 3 — `componentDidMount` runs (after the first render):

- The API call is made to JSONPlaceholder.
- It awaits the response and parses the JSON.
- It calls `this.setState({ users: usersData })`.

Step 4 — `setState` triggers the second `render()`:

- `this.state.users` now holds **10 user objects**.
- `.map()` produces **10 `<tr>` elements**; the table body is populated.
- The user sees the complete table.

Proof with `setTimeout`

To make the two-render sequence clearly visible (simulating a slow API), delay the fetch by 3 seconds:

```
async componentDidMount() {
  console.log("Inside componentDidMount of UsersList");
  setTimeout(async () => {
    try {
      const response = await fetch('https://jsonplaceholder.typicode.com/users');
      const usersData = await response.json();
      this.setState({ users: usersData });
    } catch (error) {
      console.log(error);
    }
  }, 3000); // wait 3 seconds before fetching
}
```

- For the first 3 seconds: the table body is empty (or shows "Loading...").
- After 3 seconds: `setState` fires → second render → the table fills with data.
- This makes the two-render lifecycle clearly observable.

Conditional rendering: show a loader while loading

Instead of an empty table body, show a message until data arrives:

```
<tbody>
  {
    this.state.users.length === 0
      ? (
          <tr>
            <td colSpan="5">Data is loading, please wait ... </td>
          </tr>
        )
      : this.state.users.map(user => (
          <tr key={user.id}>
            <td>{user.id}</td>
            <td>{user.name}</td>
            <td>{user.username}</td>
            <td>{user.phone}</td>
            <td>{user.email}</td>
          </tr>
        ))
  }
</tbody>
```

How it works: `condition ? componentC1 : componentC2`

- If `users.length === 0` (true on the first render) → show the loading row (C1).
- If `users.length > 0` (true on the second render, after data arrives) → show the mapped rows (C2).
- This is the **same ternary operator from JavaScript**, applied directly inside JSX.

Two render passes in plain English:

- First render: condition is true → "Data is loading, please wait..." appears.
- After `componentDidMount` fetches and calls `setState`: condition is false → the actual user rows appear.

> **Exam:** Conditional rendering = a ternary (`condition ? A : B`) inside JSX that switches between two UI states. Combined with list rendering and `componentDidMount`, this is the complete pattern for async data loading in class components.

MCP (Model Context Protocol): A Brief Aside

What is MCP?

- **MCP (Model Context Protocol)** is **similar to an API** but designed specifically to **connect AI agents (like Claude) with third-party tools and applications**.
- It lets an AI agent **read from and write to** external services automatically.

Use cases discussed

Use case	How MCP helps
Daily stand-up summary email	Claude tracks what was done and what is pending; at 6 PM every day MCP fetches the summary from Claude and sends it via Outlook to the manager — no manual daily reporting
Timesheet filling	Developers forget timesheets and the end-of-month catch-up is painful. MCP can fetch daily work summaries from Claude and auto-fill the timesheet portal date by date. Timesheets drive customer billing, so accuracy matters
PR (pull request) review on Git	MCP connects Claude to GitHub/GitLab; Claude can automatically review PRs and post comments, reducing manual review load

Human verification is still essential

- AI can automate repetitive tasks, but **human oversight is required to verify correctness**.
- This was cited as the primary reason software engineers remain relevant in the age of AI.
- Automation handles the grunt work; humans validate the output.

> **Exam:** MCP extends AI agents beyond conversation into real tool integration (email, timesheets, PR reviews) while humans stay in the loop to verify.

Combined Worked Example

This ties the day together: a Vite React app (class components **and** a function component) using props, state, events, `componentDidMount` fetch, `.map()` and conditional rendering. Create the app with `npm create vite@latest` (name `react-demo`, React, JavaScript), run `npm i`, then replace/add the files below inside `src/`.

`src/main.jsx` — entry point (boilerplate CSS import removed, StrictMode removed for single logs):

```jsx
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <App />
);
```

`src/Intro.jsx` — function component receiving props:

```jsx
export function Intro({ headingText, description, buttonText }) {
  return (
    <div>
      <h1>{headingText}</h1>
      <p>{description}</p>
      <button>{buttonText}</button>
    </div>
  );
}
```

`src/Scorecard.jsx` — function component with `useState`:

```jsx
import { useState } from 'react';

export function Scorecard() {
  const [score, setScore] = useState(0);

  const increaseScore = () => {
    setScore(score + 3);
  };

  const decreaseScore = () => {
    setScore(score - 1);
  };

  return (
    <div>
      <h2>Score (function component)</h2>
      <h1>{score}</h1>
      <button onClick={increaseScore}>+</button>
      <button onClick={decreaseScore}>−</button>
    </div>
  );
}
```

`src/Scoreboard.jsx` — class component with state and lifecycle logs:

```
import { Component } from 'react';

export class Scoreboard extends Component {
  constructor(props) {
    super(props);
    this.state = { score: 0 };
    console.log('1. Scoreboard constructor');
  }

  componentDidMount() {
    console.log('3. Scoreboard componentDidMount');
  }

  increaseScore = () => {
    this.setState({ score: this.state.score + 3 });
  }

  decreaseScore = () => {
    this.setState({ score: this.state.score - 1 });
  }

  render() {
    console.log('2. Scoreboard render');
    return (
      <div>
        <h2>Score (class component)</h2>
        <h1>{this.state.score}</h1>
        <button onClick={this.increaseScore}>+</button>
        <button onClick={this.decreaseScore}>-</button>
      </div>
    );
  }
}
```

`src/UsersList.jsx` — class component fetching in `componentDidMount`:

```
import { Component } from 'react';

export class UsersList extends Component {
  constructor(props) {
    super(props);
    this.state = { users: [] };
  }

  async componentDidMount() {
    try {
      const response = await fetch('https://jsonplaceholder.typicode.com/users');
```

```
      const usersData = await response.json();
      this.setState({ users: usersData });
    } catch (error) {
      console.log(error);
    }
  }

  render() {
    return (
      <div>
        <h4>List of Users</h4>
        <table border="1" cellPadding="10" cellSpacing="0">
          <thead>
            <tr>
              <th>ID</th>
              <th>Name</th>
              <th>Username</th>
              <th>Phone</th>
              <th>Email</th>
            </tr>
          </thead>
          <tbody>
            {
              this.state.users.length === 0
                ? (
                    <tr>
                      <td colSpan="5">Data is loading, please wait ... </td>
                    </tr>
                  )
                : this.state.users.map(user => (
                    <tr key={user.id}>
                      <td>{user.id}</td>
                      <td>{user.name}</td>
                      <td>{user.username}</td>
                      <td>{user.phone}</td>
                      <td>{user.email}</td>
                    </tr>
                  ))
            }
          </tbody>
        </table>
      </div>
    );
  }
}
```

`src/App.jsx` — the parent that decides the props and composes everything (a fragment avoids an extra wrapper):

```jsx
import { Intro } from './Intro';
import { Scorecard } from './Scorecard';
import { Scoreboard } from './Scoreboard';
import { UsersList } from './UsersList';

function App() {
  return (
    <>
      <Intro
        headingText="Welcome to our app"
        description="This is our first React app"
        buttonText="Click Me"
      />
      <Intro
        headingText="Know About Us"
        description="We have 20 years of experience in the industry"
        buttonText="Know More About Us"
      />
      <Scorecard />
      <Scoreboard />
      <UsersList />
    </>
  );
}

export default App;
```

Run with `npm run dev` and open `localhost:5173`.

End-to-end trace

1. The browser loads the single `index.html`; `main.jsx` runs first and mounts `App` into the root element.
2. `App` returns a fragment containing five component tags (`Intro` twice, `Scorecard`, `Scoreboard`, `UsersList`) — React calls each component to get its UI.
3. Both `Intro` tags use the **same component** but different props supplied by `App`; `Intro` only reads them.
4. `Scorecard` calls `useState(0)` → `score = 0`. Clicking `+` calls `setScore(score + 3)`: the value changes **and** the component re-renders — only the changed `<h1>` is updated in the real DOM thanks to the virtual DOM diff.
5. `Scoreboard` logs `1. constructor`, `2. render`, `3. componentDidMount` on load. Clicking `+` logs only `2. Scoreboard render`.
6. `UsersList`: the constructor sets `users: []`; the first render shows "Data is loading, please wait..." (condition true); `componentDidMount` fetches 10 users and calls `setState`; the second

render takes the `.map()` branch and shows 10 rows.

7. Because `key={user.id}` is unique per row, React can track each row across renders.

Quick Revision Checklist

Topic	One-line takeaway	Topic	One-line takeaway
`npm create vite@latest`	Scaffold; then `npm i` and `npm run dev` → `localhost:5173`	Boilerplate cleanup	Default CSS is Vite's; empty `App.jsx`, remove `index.css` import in `main.jsx`
Class component	`extends Component` + `render()` + export	Function component	Plain function returning JSX; lightweight, preferred today
Using a component	Tag name = class/function name exactly (`<Navbar />`)	Import path	No `.jsx` extension needed
JSX	JavaScript + XML; HTML-like tags inside JS	XML vs HTML	XML has user-defined tags; HTML has predefined tags
Close every tag	` `, `<input />` — XML rule inherited by React	Single parent	One root element; wrap in `<div>` or fragment `<> … </>`
Props	Attributes of a component; parent decides, read-only	Props in function / class	Argument (destructure) / `this.props`
`{}` in JSX	Embeds a JS value; without it the name prints as text	State	Owned, initialised and changed by the component itself
Class state	`this.state` in constructor; update with `this.setState()`	`setState`	Updates memory **and** triggers a re-render
Function state	`const [x, setX] = useState(init)`	Direct mutation	`this.state.x = …` — no re-render, UI stale
Event props	camelCase: `onClick`, `onKeyUp`	Handler reference	`onClick={fn}` not `onClick={fn()}` ; `this.fn` in classes

Topic	One-line takeaway	Topic	One-line takeaway
Virtual DOM	Lightweight copy of DOM; diff old vs new, update only changes	`react-dom`	Translates virtual DOM into the real browser DOM
Hooks	Regular JS functions starting with `use`	Common hooks	`useState`, `useEffect`, `useMemo`, `useCallback`, `useNavigate`, `useParams`
Lifecycle phases	Mounting (appear) and Unmounting (removed)	Order	`constructor` → `render` → `componentDidMount`
`componentDidMount`	Runs once after the first render; not on re-renders	State update	Only `render()` runs again
`componentWillUnmount`	Runs once on removal	StrictMode	Double-invokes in dev only; remove from `main.jsx` for single logs
Fetch in constructor	Blocks render → blank screen	Fetch in `render()`	Side effects / infinite loops
Fetch in `componentDidMount`	UI shows first, data loads in background, `setState` re-renders	State as bridge	Shares data between lifecycle methods; `const` is block-scoped
List rendering	`.map()` data into an array of JSX; React renders it	`key` prop	Unique ID per item for React to track
Conditional rendering	Ternary `cond ? A : B` inside JSX	Two renders	Loader first, data after `setState`
`useEffect`	Function-component lifecycle, next session	MCP	Connects AI agents to tools; humans still verify

Practice Exercises

Concept

1. List the three mandatory steps to create a class component.
2. Why is a plain JavaScript class not a React component? What does `extends Component` provide?

3. What is JSX? Why is the file extension `.jsx`?
4. Compare HTML and XML, and explain why `<Navbar />` is called an XML-style tag.
5. State the two JSX rules (closing tags, single parent). Give two ways to satisfy the single-parent rule, and the difference between them.
6. What are props? How are they different from state? Give the two key rules about props.
7. How does a function component receive props, and how does a class component?
8. What does `setState` do? Why is `this.state.score = 5` wrong?
9. What does `useState` take and return? What is the naming convention for the setter?
10. What is the Virtual DOM and why does it make React faster than direct DOM manipulation?
11. What is a hook? What is the naming convention and is it a hard rule?
12. Define mounting and unmounting. Which lifecycle function runs after the first render only?
13. Why is the constructor the wrong place for an API call? Why is `render()` also wrong?
14. Why is `componentDidMount` the right place? Name two real-world apps from the lecture that show a skeleton before data.
15. Why can `render()` not use a `const` declared inside `componentDidMount`, and how does state solve it?
16. What does React need in order to render a list? What does `.map()` return?
17. What is MCP? Name the three use cases from the lecture and why humans are still needed.

Application

18. Write a function component `Greeting` that receives a `name` prop (destructured) and renders `<h2>Hello, {name}</h2>`, then show how `App` would use it for "Asha".
19. Write a class component `Counter` with state `count` starting at 0, and a button that increases it by 1 using `setState`.
20. Rewrite `Counter` from Q19 as a function component using `useState`.
21. Given `this.state = { fruits: ['Apple', 'Mango', 'Banana'] }`, write the JSX inside `render()` that displays each fruit in a `<li>` inside a `<ul>`.
22. Write the JSX expression that shows `<p>Loading …</p>` while `this.state.items.length === 0`, otherwise a `<p>` with the number of items.
23. Write the commands to create a Vite React app, install dependencies and start it. At which address does it run?

Predict the output / behaviour

24. For the `Scoreboard` class with logs `1. Constructor`, `2. Render`, `3. componentDidMount`, what is printed on first load? What is printed after clicking `+` twice? (Assume StrictMode removed.)

25. With `React.StrictMode` present in development, why might each log appear twice?
26. In `UsersList`, what does the user see (a) after the first render, (b) after `setState` completes?
27. In the `UsersList` with the 3-second `setTimeout`, what happens during the first 3 seconds and after?
28. `this.state.myList = [10, 20, 30]` and render does `this.state.myList.map(v => <p>{v}</p>)`. What is produced?

Spot the bug

29. `return ( <h1>Title</h1> <p>Text</p> );` — what error occurs and what are two fixes?
30. `<input type="text">` inside JSX — what is wrong?
31. `<button onClick={this.increaseScore()}>+</button>` — what is wrong?
32. `increaseScore = () => { this.state.score = this.state.score + 3; }` — what is wrong?
33. `<h1>score</h1>` is meant to display the state `score` (function component) but the page shows the word "score". Why?
34. A developer calls `fetch( ... )` inside `render()`. What problems arise?
35. A component calls `const usersData = ...` inside `componentDidMount` and then uses `usersData` inside `render()`. What error is produced and what is the fix?
36. `class Navbar { render() { return <div>Menu</div>; } }` is used as `<Navbar />`. Why does it not work?

Practice Answers

1. (1) Define a class and `extend Component`; (2) define a `render()` function that returns the UI; (3) export the class.

2. A plain class has no React abilities. `extends Component` inherits `render` handling, `this.props`, `this.state` / `setState` and the lifecycle functions (`componentDidMount`, etc.).

3. JSX = JavaScript + XML: it lets you write HTML-like code directly inside JavaScript, which a plain `.js` file cannot. The extension is `.jsx` because React uses XML-style user-defined tags (`<Navbar />`) behind the scenes.

4. HTML has predefined tags and displays data; XML has only user-defined tags and was used to store data (before JSON). `<Navbar />` is not a built-in HTML tag but a user-defined tag naming your component, which is XML-style.

5. Rule 1: every tag must be closed (`<br />` , `<input />` , or `<br></br>`). Rule 2: JSX must have a single parent. Fix with a wrapper `<div>` (adds an extra DOM element) or an empty fragment `<> … </>` (adds no DOM node).

6. Props are the attributes of a component, supplied by the parent; state is data the component owns and changes itself. Rules: (1) a component cannot decide the value of its own props — the parent does; (2) a component should not change its props — they are read-only.

7. A function component receives one props object as the function argument (often destructured). A class component receives it as `this.props` , inherited from `Component` . Passing props in JSX is identical for both.

8. `setState` updates the value in memory **and** triggers a re-render so the UI shows the new data. `this.state.score = 5` changes memory only; React is not notified, so no re-render occurs and the UI stays stale.

9. `useState` takes the initial value and returns an array: `[currentValue, updaterFunction]` . For state `score` , the setter is `setScore` .

10. The Virtual DOM is a lightweight copy of the real DOM. On a state update React builds a new snapshot, diffs it with the previous one, and updates only the changed elements in the real DOM — whereas direct DOM manipulation recreates/repaints the whole tree for any change.

11. A hook is a regular JavaScript function that brings class features (state, lifecycle) into function components. Names should start with `use` ; this is a React recommendation, not a hard syntax rule.

12. Mounting: the component is displayed on screen for the first time. Unmounting: it is removed from the screen. `componentDidMount` runs once, right after the first render.

13. The constructor runs before `render()` , so an API call there blocks the render and the user sees a blank screen (not even a loader) for as long as the API takes. `render()` must be a pure description of the UI; fetching there creates side effects, repeats on every re-render and can cause infinite loops.

14. `componentDidMount` runs after the first render has painted the screen, so the user already sees static content or a loader/skeleton while the fetch happens in the background; `setState` then re-renders with data. Examples: Instagram (feed skeleton), YouTube (video card skeletons), Facebook (timeline skeletons), WhatsApp Web (chat list shimmer).

15. `const` / `let` are block-scoped, so `usersData` exists only inside `componentDidMount` 's block. State lives on the instance (`this.state`), so `componentDidMount` writes it with `setState` and `render()` reads `this.state.users` .

16. React needs an **array of JSX elements**. `.map()` returns a new array where each data item has been turned into a JSX element; React renders it automatically.

17. MCP (Model Context Protocol) is like an API designed to connect AI agents such as Claude to third-party tools. Use cases: daily stand-up summary email via Outlook at 6 PM, auto-filling timesheets, and automatic PR review comments on GitHub/GitLab. Humans are still needed to verify the correctness of the automated output.

18. Component and usage:

```jsx
// Greeting.jsx
export function Greeting({ name }) {
  return <h2>Hello, {name}</h2>;
}

// In App:
<Greeting name="Asha" />
```

19. Class component:

```jsx
import { Component } from 'react';

export class Counter extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  increase = () => {
    this.setState({ count: this.state.count + 1 });
  }

  render() {
    return (
      <div>
        <h1>{this.state.count}</h1>
        <button onClick={this.increase}>+1</button>
      </div>
    );
  }
}
```

20. Function component:

```jsx
import { useState } from 'react';

export function Counter() {
  const [count, setCount] = useState(0);

  const increase = () => {
```

```
    setCount(count + 1);
  };

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={increase}>+1</button>
    </div>
  );
}
```

21. JSX inside `render()`:

```
<ul>
  {
    this.state.fruits.map(fruit => (
      <li key={fruit}>{fruit}</li>
    ))
  }
</ul>
```

22. Conditional expression:

```
{
  this.state.items.length === 0
    ? <p>Loading ... </p>
    : <p>{this.state.items.length} items</p>
}
```

23.

```
npm create vite@latest
npm i
npm run dev
```

It runs at `localhost:5173`.

24. First load: `1. Constructor`, `2. Render`, `3. componentDidMount`. After clicking `+` twice: `2. Render` printed twice (once per click); constructor and `componentDidMount` do not run again.

25. `React.StrictMode` in `main.jsx` intentionally double-invokes lifecycle methods in development to expose side effects. It is development-only; production runs each once. Remove StrictMode to see single logs.

26. (a) After the first render: the table header and the "Data is loading, please wait..." row (with the plain version, an empty tbody). (b) After `setState`: the second render shows 10 user rows.

27. For 3 seconds the table shows the loading row (empty body without a loader). After 3 seconds the fetch finishes, `setState` fires, the second render runs and the table fills with data.

28. An array of elements: `[ <p>10</p>, <p>20</p>, <p>30</p> ]`, which React renders as three paragraphs (React warns about a missing `key`).

29. "JSX expressions must have one parent element" — two siblings at the same level. Fix 1: wrap in `<div> … </div>`. Fix 2: wrap in a fragment `<> … </>`.

30. The tag is not closed. In JSX it must be `<input type="text" />`.

31. The brackets call the function immediately while rendering instead of on click. Pass the reference: `onClick={this.increaseScore}`.

32. It mutates state directly; React is not told, so there is no re-render and the UI stays stale. Use `this.setState({ score: this.state.score + 3 })`.

33. Without curly braces `score` is literal text. Write `<h1>{score}</h1>`.

34. `render()` should be pure. A fetch there creates side effects and runs on every re-render; fetching then calling `setState` re-renders again and again, causing an infinite loop. Fetch in `componentDidMount`.

35. `ReferenceError` — `usersData` is block-scoped to `componentDidMount`. Fix: `this.setState({ users: usersData })` there, and read `this.state.users` in `render()`.

36. It does not extend `Component`, so it is just a plain class with no React behaviour. It needs `class Navbar extends Component` (with `import { Component } from 'react'`) and an export.